

The PiQuadrupole Calculator

Generated/Reviewed Documentation (Codex/Opus/Human) for v0.0.1

1 What the calculator computes

The PiQuadrupole calculator describes one electric effect of an aromatic ring: the quadrupolar field of its π -electron cloud. In a multipole description, a quadrupole is the rank-2 part of a localized charge distribution: it records that charge is arranged more along some directions than others after the monopole and dipole pieces have been set aside. For an aromatic ring, the model places an axial quadrupole at the ring center and aligns its axis with the ring normal. The calculator evaluates the geometric field-gradient pattern from that source at each nearby atom.

The main output is a rank-2 tensor with units $\text{\AA}^{-5}$. It is an electric-field-gradient kernel, not a shielding in ppm. The physical route to shielding is the Buckingham response: the electronic shielding at a nucleus can change in an electric field and in an electric field gradient. The code stores the geometry that a calibrated coefficient can later multiply. It also exports a separate scalar with units $\text{\AA}^{-4}$, the angular r^{-4} field kernel used for an isotropic Buckingham-style term. Before calibration, both outputs are geometric descriptors whose changes may correlate with a DFT shielding contribution; they are not chemical shifts.

2 Where shielding enters

For a nucleus in an applied magnetic field B_0 , the local field is written

$$B_{\text{nuc}} = (\mathbf{I} - \boldsymbol{\sigma})B_0. \quad (1)$$

$\mathbf{I}$ is the 3×3 identity matrix, and $\boldsymbol{\sigma}$ is the nuclear shielding tensor. In physical NMR, $\boldsymbol{\sigma}$ is dimensionless: it is the linear coefficient relating the applied magnetic field to the secondary field produced by the electrons at the nucleus. A matrix is needed because the secondary field depends on molecular orientation. A field applied along z , for example, can induce a local response with x , y , and z components when the molecule is held in a fixed frame.

This is a quantum-mechanical response. The applied magnetic field perturbs the electronic state, and the perturbed electrons generate the shielding field seen by the nucleus. The PiQuadrupole calculator does not solve that response. It supplies a classical geometric input for one perturbation that can affect the response: an electric field gradient produced by nearby aromatic π -electron density.

The connection is through a Taylor expansion of shielding in the local electric environment. A scalar electric field term can change the isotropic shielding, while an electric field gradient is already a rank-2 object and can couple naturally to the orientation-dependent part of $\boldsymbol{\sigma}$. In the usual Buckingham notation, coefficients such as A and γ carry the chemical response. This implementation leaves those coefficients outside the C++ kernel and records the geometry that calibration can compare with DFT shielding tensors.

The solution observable most often quoted is an isotropic chemical shift, obtained from the orientationally averaged shielding after referencing and sign convention. That scalar is not the whole tensor. This calculator is used here because its field-gradient kernel has five anisotropic components, so it can test whether ring quadrupole geometry carries angular shielding signal beyond a scalar proximity measure.

3 The tensor split used by the code

The C++ type `SphericalTensor` decomposes a real 3×3 tensor $\mathbf{X}$ into three parts. The isotropic component is

$$T_0 = \frac{1}{3} \text{tr } \mathbf{X}, \quad (2)$$

the average of the three diagonal entries. It is the part that multiplies $\mathbf{I}$ and survives rotational averaging. The antisymmetric component is packed as the pseudovector

$$\mathbf{T}_1 = \frac{1}{2} \begin{pmatrix} X_{yz} - X_{zy} \\ X_{zx} - X_{xz} \\ X_{xy} - X_{yx} \end{pmatrix}. \quad (3)$$

This stores the part of the matrix that changes sign on transpose. It is usually not observed directly in standard isotropic solution NMR, but the storage format keeps it because some calculator tensors can have such a part.

The remaining part is the symmetric traceless matrix

$$\mathbf{S} = \frac{\mathbf{X} + \mathbf{X}^\top}{2} - T_0 \mathbf{I}, \quad \text{tr } \mathbf{S} = 0. \quad (4)$$

It has five independent entries. The code writes those entries in an isometric real-spherical basis,

$$\mathbf{T}_2 = \begin{pmatrix} \sqrt{2} S_{xy} \\ \sqrt{2} S_{yz} \\ \sqrt{3/2} S_{zz} \\ \sqrt{2} S_{xz} \\ (S_{xx} - S_{yy})/\sqrt{2} \end{pmatrix}. \quad (5)$$

The normalization in Eq. (5) preserves the Frobenius norm: $\sum_m T_{2,m}^2 = \sum_{ab} S_{ab}^2$. Thus a dot product between the five stored T_2 numbers is the Frobenius dot product between the underlying traceless-symmetric matrices.

The PiQuadrupole field-gradient kernel is symmetric and traceless away from the ring source. After decomposition, the main tensor output therefore has $T_0 = 0$ and $\mathbf{T}_1 = \mathbf{0}$ analytically, with the signal in T_2 . The export still uses the nine-number layout

$$[T_0, T_{1x}, T_{1y}, T_{1z}, T_{2,-2}, T_{2,-1}, T_{2,0}, T_{2,+1}, T_{2,+2}]$$

because that is the shared tensor format. The separate `pq_per_type_T0` array is not T_0 of this tensor; it is the per-ring-type sum of the scalar r^{-4} Buckingham A-term kernel.

4 The method implemented

For each aromatic ring, the geometry code stores a center $\mathbf{c}$, a unit normal $\hat{\mathbf{n}}$, and an average radius R . The center is the mean of the ring-atom positions. The normal is found by an SVD plane fit: the singular vector with the smallest spread is perpendicular to the best-fit plane, and its sign is fixed by the first two ring edges. The radius is the mean distance from the center to the ring vertices. The aromatic ring types are PHE, TYR, TRP6, TRP5, TRP9, HIS, HID, and HIE; the saturated proline ring is not included in the aromatic ring count used here.

For a target atom at $\mathbf{r}_i$, define

$$\mathbf{d} = \mathbf{r}_i - \mathbf{c}, \quad r = |\mathbf{d}|, \quad h = \mathbf{d} \cdot \hat{\mathbf{n}}, \quad \cos \theta = \frac{h}{r}. \quad (6)$$

$\mathbf{d}$ and r are measured in Å. The scalar h is the signed height of the atom above the ring plane, also in Å, and θ is the angle from the ring normal to the ring-center-to-atom direction.

The physical axial-quadrupole potential motivating the calculation has the form

$$\Phi(\mathbf{d}) = \frac{\Theta}{2} \frac{3 \cos^2 \theta - 1}{r^3}, \quad (7)$$

where Θ is a quadrupole strength. The electric field gradient is the negative Hessian of this potential,

$$V_{ab} = -\frac{\partial^2 \Phi}{\partial d_a \partial d_b}. \quad (8)$$

The Hessian is the matrix of local curvatures of a scalar field. In the quadratic Taylor model $\Phi(\mathbf{d} + \mathbf{x}) \approx \Phi(\mathbf{d}) + \mathbf{g} \cdot \mathbf{x} + \frac{1}{2} \mathbf{x}^\top \mathbf{H} \mathbf{x}$, the gradient $\mathbf{g}$ gives the local electric field direction and the Hessian $\mathbf{H}$ gives how that field changes with displacement. Outside a charge source, Laplace's equation says $\text{tr } \mathbf{H} = \nabla^2 \Phi = 0$. In curvature language, a positive curvature in one direction must be balanced by negative curvature in another. That is the direct reason this field-gradient tensor is traceless and lands in the T_2 subspace.

Here Θ denotes a physical quadrupole moment, with charge times length-squared units before electrostatic constants are chosen. It is shown to locate the formula in the usual physics; it is not a parameter used by the C++ calculator.

The source uses Stone's interaction tensor to evaluate the needed fourth derivatives of $1/r$:

$$T_{abcd} = \frac{\partial^4 (1/r)}{\partial d_a \partial d_b \partial d_c \partial d_d}.$$

The code defines the exported tensor as the bare geometric contraction

$$G_{ab} = T_{abcd} \hat{n}_c \hat{n}_d, \quad (9)$$

where the c and d indices are summed over x, y, z . A scaled physical EFG would include a prefactor such as $-\Theta/2$. The C++ result does not apply that sign or strength; it exports G and leaves scaling to calibration. Written in the variables of Eq. (6), the implemented kernel is

$$G_{ab} = \frac{105h^2 d_a d_b}{r^9} - \frac{30h(\hat{n}_a d_b + \hat{n}_b d_a)}{r^7} - \frac{15d_a d_b}{r^7} + \frac{6\hat{n}_a \hat{n}_b}{r^5} + \delta_{ab} \left(\frac{3}{r^5} - \frac{15h^2}{r^7} \right). \quad (10)$$

δ_{ab} is one for equal Cartesian indices and zero otherwise. Each term has units $\text{\AA}^{-5}$. Although Eq. (10) contains powers through r^{-9} , the factors $h^2 d_a d_b$, $h d_b$, and $d_a d_b$ carry powers of length; the far-field tensor decays as r^{-5} , as expected for an electric field gradient from a quadrupole.

The same kernel helper also computes

$$q_i = \frac{3 \cos^2 \theta - 1}{r^4}. \quad (11)$$

This scalar has units $\text{\AA}^{-4}$. It is stored separately because it represents the geometric part of a Buckingham electric-field term, not the field-gradient tensor in Eq. (10).

For each atom, the calculator asks the spatial index for aromatic rings within $15.0 \text{\AA}$, the ring-current spatial cutoff in the parameter TOML. Candidate pairs are rejected when $r < 0.1 \text{\AA}$, the singularity guard distance, because the inverse-power kernel diverges near the source. A near-field filter also requires $r > 0.5(2R) = R$; the near-field exclusion ratio is 0.5, and the source extent passed to the filter is the ring diameter. A topology filter rejects atoms that are ring vertices or covalently bonded to a ring vertex, since those atoms are in a through-bond regime rather than a through-space multipole regime.

For accepted rings, the calculator stores the per-ring tensor G , its `SphericalTensor` decomposition, and the scalar q_i on the atom's ring-neighbour record. It accumulates

$$G_i^{\text{tot}} = \sum_{\rho \in \mathcal{A}_i} G^{(i\rho)}, \quad (12)$$

where $\mathcal{A}_i$ is the accepted ring set for atom i . The total is then decomposed with Eqs. (2)–(5). In parallel, the code sums q_i by aromatic ring type and sums the five T_2 components of each accepted per-ring tensor

by the same type. The TOML ring-current intensities and Johnson–Bovey lobe offsets are not used in this calculation. Ring type changes which output bin receives the unscaled geometry; it does not multiply the kernel by a type-specific quadrupole strength in the C++ code. The source position is the ring center itself; no above-plane or below-plane height offset is applied to the quadrupole source.

The grid-sampling method follows the same kernel formula at an arbitrary point in space. Since a grid point has no atom index, the topology filter is not available there. The sampling path applies the 0.1 Å singularity guard, skips points inside the average ring radius, skips rings beyond 15.0 Å, sums the remaining G tensors, and returns their decomposition.

5 Inputs and output

The direct inputs are molecular geometry and ring topology: atom positions in Å, the typed aromatic ring list, ring vertex positions, and a spatial index used to find nearby rings. The declared runtime dependencies are `GeometryResult` and `SpatialIndexResult`.

The calculator does not consume MOPAC charges, AIMNet2 charges or embeddings, ff14SB charges, APBS fields, or DSSP assignments. Those data sources are parallel inputs to other feature channels: MOPAC supplies semiempirical charges and bond orders, AIMNet2 supplies neural-network charges and embeddings, ff14SB supplies force-field charges for electrostatics, APBS supplies a Poisson–Boltzmann electric field and field gradient, and DSSP supplies secondary-structure context. Their internal methods are outside this calculator.

`WriteFeatures` emits three arrays. `pq_shielding.npy` has shape $N \times 9$ and stores the packed `SphericalTensor` for G_i^{tot} in $\text{\AA}^{-5}$; despite the filename, it is the unscaled quadrupole EFG kernel. `pq_per_type_T0.npy` has shape $N \times 8$ and stores the per-aromatic-type scalar sums from Eq. (11) in $\text{\AA}^{-4}$. `pq_per_type_T2.npy` has shape $N \times 40$, eight aromatic ring types times five T_2 components, in $\text{\AA}^{-5}$. Calibration against DFT reference tensors is what can turn these geometric quantities into shielding contributions; the raw arrays themselves are not shielding predictions.